

Regression, Classification & Loss Functions

KAUST–Mawhiba IOAI Training

Week 1 · Day 3



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

April 25, 2026

1. Recap & The ML Toolbox
2. Regression: MSE, MAE, Linear Regression, Gradient Descent
3. Classification: Cross-Entropy, Logistic Regression, Sigmoid & Softmax
4. Evaluation: Loss vs. Metric, Confusion Matrix
5. Precision, Recall, F1-Score, Baselines
6. ROC Curves & AUC

In Day 2 we introduced the **Three Pillars of Learning**:

1. **Model** (f_{θ})

Maps inputs
to predictions

2. **Loss** (J)

Measures prediction error

3. **Optimizer**

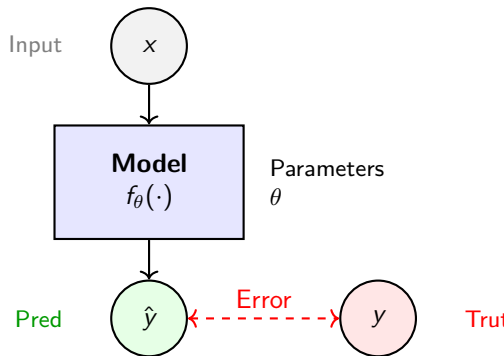
Updates parameters

Today we make these **concrete**: we implement all three pillars for **Regression** and **Classification**, and learn how to **evaluate** the results.

Definition

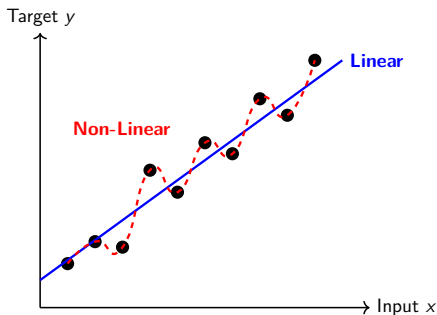
A **Model** (f_{θ}) is a simplified mathematical representation of reality. It maps inputs (x) to outputs ($\hat{y}$) using learned parameters (θ).

The Goal: Find the “best” function f that minimizes the error between prediction $\hat{y}$ and reality y .



The relationship between inputs x and target y varies wildly. Choosing the right model family matters to avoid **Underfitting** (too simple) or **Overfitting** (too complex).

- ▶ **Simple:** Linear. (e.g., *House Size* $\rightarrow$ *Price*)
- ▶ **Complex:** Non-linear. (e.g., *Image Pixels* $\rightarrow$ "Cat")



Machine Learning Models (f_θ)

1. Linear Models
Linear Regression
Logistic Regression

↑ **Today!**

2. Distance-Based
KNN
(Non-Parametric)

3. Kernel-Based
SVM

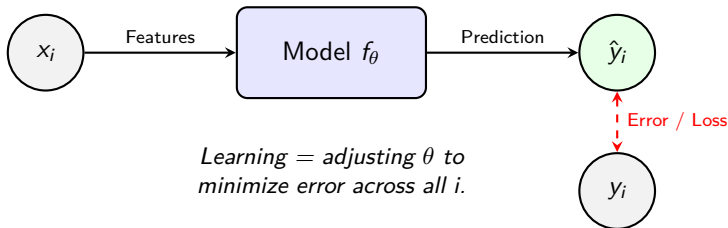
4. Tree-Based
Decision Trees
Random Forest
Gradient Boosting

The Objective

Given a labeled dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, learn a mapping f that generalizes to unseen data:

$$\hat{y} = f(x; \theta) \approx y$$

where θ represents the learnable model parameters.



Definition: The target y is **Continuous** ($y \in \mathbb{R}$).

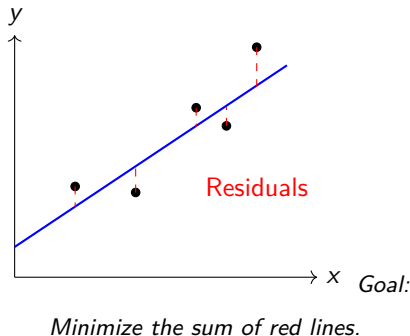
Examples:

- ▶ House Prices (\$300k, \$500k)
- ▶ Temperature (24.5° C)
- ▶ Stock Value

The Loss: Mean Squared Error

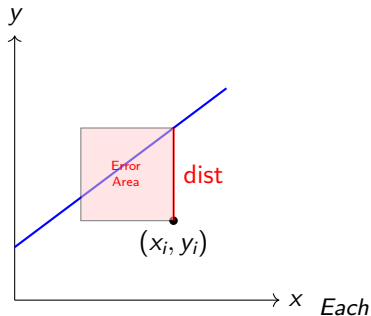
Measures the squared distance between points and the line.

$$\text{MSE} = \frac{1}{N} \sum (y - \hat{y})^2$$



The 3 Steps to MSE Loss:

1. **Residuals** ($y - \hat{y}$): measures the raw distance. Points below the line give **negative** distances.
2. **The Square** ($\dots$)²: Squaring ensures all distances are positive so they don't cancel out. It also heavily penalizes **outliers**.
3. **The Average** ($\frac{1}{N} \sum$): We aggregate errors from all N samples to get a single performance score.

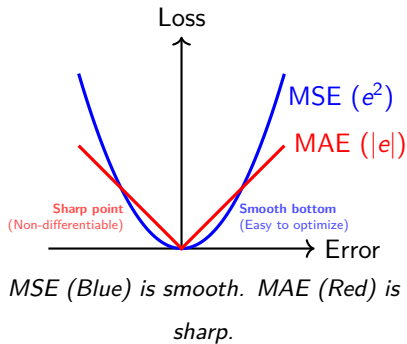


"error" acts as a square area we want to shrink to zero.

Note

If we use the **Absolute** difference $|y - \hat{y}|$ instead of the square, we get **MAE**. It is more robust to outliers, but **not differentiable** at 0.

Mathematically, you can still optimize functions with **finitely** many non-differentiable points using subgradients, though this adds some complexity.

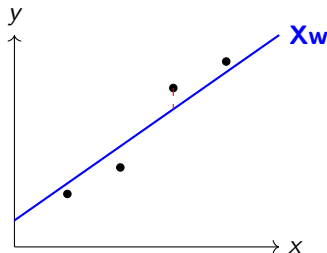


We start with the simplest assumption:
The target y is a **linear combination** of the input features.

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

The Components:

- ▶ $\hat{y}$: predicted value
- ▶ $x_1, \dots, x_d$: the input features.
- ▶ $w_1, \dots, w_d$: the weights. How much each feature matters.
- ▶ w_0 : the bias (Intercept).



Step 1: The Bias Trick: We add a “dummy” feature $x_0 = 1$ to the input so we have each weight multiplied by a x feature.

$$\hat{y} = w_0(1) + w_1x_1 + \cdots + w_dx_d$$

Step 2: Vector Notation (For one sample): We rewrite this equation in a vector dot product form:

$$\hat{y} = \begin{bmatrix} w_0 & w_1 & \cdots & w_d \end{bmatrix} \cdot \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} = \mathbf{w}^T \mathbf{x}$$

Step 3: Matrix Notation (For all data): For N samples, prediction becomes a simple Matrix-Vector multiplication:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

Now we have the equation. But how do we choose the best $\mathbf{w}$?

We need the loss and the optimizer!

Let's wrap our model in the Mean Squared Error:

$$J(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

How do we find the $\mathbf{w}$ that minimizes this $J(\mathbf{w})$?

Option A: Analytical
(Solve for zero gradient)

OR

Option B: Iterative
(Gradient Descent)

Method A: The Normal Equation

For linear models, MSE is **convex** — guaranteed global minimum!

Step 1: The Gradient (derived via Chain Rule):

$$\nabla J(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Step 2: Set to Zero

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

Step 3: Solve for $\mathbf{w}$ Multiply both sides by inverse $(\mathbf{X}^T \mathbf{X})^{-1}$:

The Normal Equation

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Cons: Inverting large matrices is very slow ($O(d^3)$). Doesn't work if X is singular.

Instead of jumping to the solution, we take small steps.

Step 1: Calculate Gradient

$$\nabla J(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Step 2: Update Rule Move opposite to the gradient to reduce error.

Iterative Update

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \alpha \nabla J$$

α : Learning Rate (Step size)

Pros: Very efficient for large datasets.

Definition:

Target y is **discrete**: $y \in \{0, 1\}$ (e.g., spam/not spam)

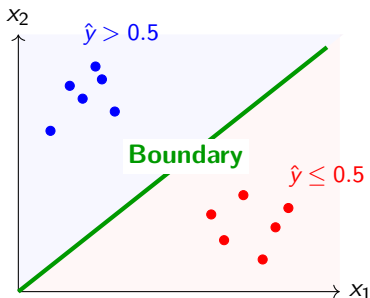
How It Works:

1. Model outputs **probability**:
 $\hat{y} \in [0, 1]$
2. Apply **threshold** (usually 0.5) to classify
3. If $\hat{y} \geq 0.5 \rightarrow$ Class 1, else Class 0

Example

$\hat{y} = 0.87 \xrightarrow{\text{threshold } 0.5} \text{Class 1 (Spam)}$

Decision Boundary Visualization



Goal: Find optimal boundary that maximizes class separation

Problem with MSE

MSE measures **numeric distance**, but in classification, we care about **probability confidence**, not distance!

What We Really Want

A loss function that **heavily penalizes confident wrong predictions**

Example: Classifying an image as a dog with 98% confidence when it's actually a cat should incur massive penalty!

Solution: The Logarithm!

The logarithm function has exactly the property we need:
it grows exponentially as confidence in the **wrong** class increases

Intuition (True Label $y = 1$):

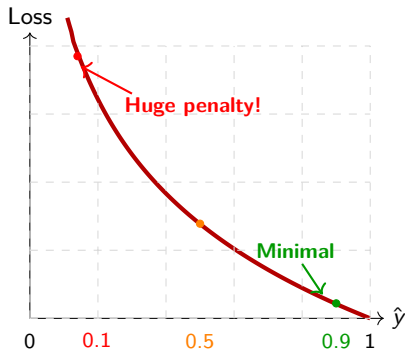
Model outputs probability $\hat{y}$ for class 1

Log Penalty = $-\log(\hat{y})$ behaves as:

- ▶ $\hat{y} = 0.99 \rightarrow \text{Loss} \approx 0.01$ **Great!**
- ▶ $\hat{y} = 0.50 \rightarrow \text{Loss} \approx 0.69$ **Uncertain**
- ▶ $\hat{y} = 0.01 \rightarrow \text{Loss} \approx 4.6$ **Disaster!**

The same concept applies when the true label is 0.

Loss vs. Predicted Probability (when true label = 1)



Curve explodes as $\hat{y} \rightarrow 0$
Forces model to be cautious!

Loss Function: Binary Cross-Entropy (Log Loss)

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

where: $y_i \in \{0, 1\}$ = true label, $\hat{y}_i \in (0, 1)$ = predicted probability, $\mathbf{w}$ = model weights.

- ▶ When $y_i = 1$: Loss = $-\log(\hat{y}_i)$ (penalizes low probability for class 1)
- ▶ When $y_i = 0$: Loss = $-\log(1 - \hat{y}_i)$ (penalizes high probability for class 1)
- ▶ Both terms encourage the model to predict probabilities close to the true labels.

Binary Classification:

- ▶ $y \in \{0, 1\}$
- ▶ Two classes only
- ▶ Example: Spam / Not Spam

Multiclass Classification:

- ▶ $y \in \{0, 1, 2, \dots, K - 1\}$
- ▶ K classes ($K > 2$)
- ▶ Example: Dog / Cat / Bird

Example: Image Classification

Input: Image $\rightarrow$ Output: $\hat{\mathbf{y}} = [0.1, 0.7, 0.2]$ for 3 classes

Class probabilities: Dog (10%), Cat (70%), Bird (20%)

Model now outputs a **probability distribution** over all K classes instead of a single probability

Loss Function: Cross-Entropy

Generalization of binary cross-entropy to K classes

$$J(\mathbf{w}) = -\frac{1}{n} \underbrace{\sum_{i=1}^n}_{\text{samples}} \underbrace{\sum_{k=1}^K}_{\text{classes}} y_{i,k} \log(\hat{y}_{i,k})$$

where $y_{i,k} \in \{0, 1\}$ is 1 if sample i belongs to class k , 0 otherwise

Example

True class: Cat (class 2), Prediction: $[0.1, 0.7, 0.2]$

Loss = $-\log(0.7) \approx 0.36$ (only the true class matters!)

The Challenge: We want to classify data (e.g., Spam vs Not Spam).

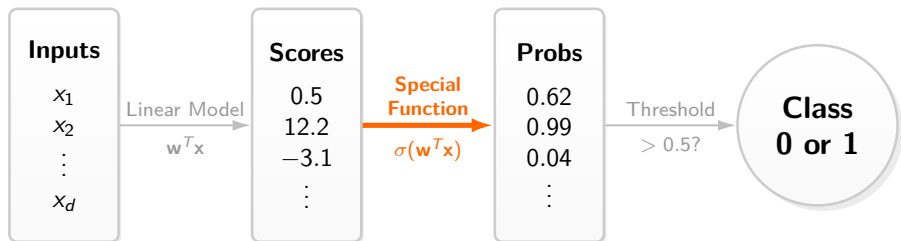
However, our Linear Model $f(x) = \mathbf{w}^T \mathbf{x}$ outputs numbers in $(-\infty, \infty)$.

Problem

If model outputs 500 or -20 , how do we interpret that as a probability?

We need a mechanism to map these raw “scores” into a valid probability range $[0, 1]$.

The Classification Pipeline



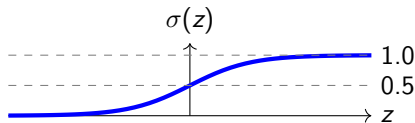
The “Special Function” we need is the **Sigmoid** (Logistic Function).

Sigmoid Function

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- ▶ Squashes any z into $(0, 1)$.
- ▶ $z = 0 \implies \hat{y} = 0.5$.
- ▶ Differentiable everywhere (Smooth).



To train, we minimize Binary Cross-Entropy:

$$J = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Compute $\frac{\partial J}{\partial w}$ using the Chain Rule:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

The sigmoid derivative perfectly cancels the denominator of the log loss:

$$\frac{\partial J}{\partial w} = \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot \hat{y}(1 - \hat{y}) \cdot x$$

Final Gradient Update

$$\nabla J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y})$$

Generalization: Multiclass (Softmax)

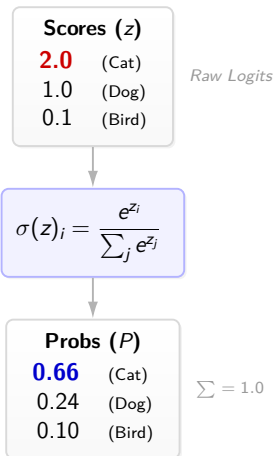
What if we have $K > 2$ classes? (e.g., Cat, Dog, Bird). We generalize Sigmoid to **Softmax**.

Softmax Function

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- ▶ **Exponent (e^z):** Forces all scores to be positive.
- ▶ **Sum ($\sum$):** Normalizes so all probabilities sum to 1.0.
- ▶ **Loss:** Cross-Entropy.

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$



Advantages

- ▶ **Interpretable:** The weights $\mathbf{w}$ directly tell us feature importance.
- ▶ **Fast & Efficient:** Training is simple matrix math. Inference is just a dot product.

Disadvantages

- ▶ **High Bias (Underfitting):** Cannot model complex, non-linear relationships.
- ▶ **Sensitive to Outliers.** Especially when using MSE.
- ▶ **Scaling Required:** Sensitive to feature magnitude (Requires Standardization).

Definition

A **Metric** is a quantifiable measure used to evaluate how well a model performs on a specific task.

Loss (For the Machine)

- ▶ Used during **Training**
- ▶ Must be **Differentiable** (smooth) so we can compute gradients
- ▶ *Examples: Cross-Entropy, MSE*

Metric (For the Human)

- ▶ Used during **Evaluation** (Val / Test)
- ▶ Must be **Interpretable** by humans (business value)
- ▶ *Examples: Accuracy, F1, RMSE*

Metrics differ based on the task: Regression (continuous) vs. Classification (discrete).

Scenario: You trained a model to predict **House Prices**.

$$\text{MAE} = 10,000$$

What it means:

- ▶ “On average, my prediction is off by **\$10,000.**”

Easy to explain.

$$\text{MSE} = 100,000,000$$

What it means:

- ▶ The unit is “Dollars Squared” ($\2)
- ▶ Hard to interpret directly!
- ▶ **Why use it?** Forces model to fix big mistakes first.

Common Fix: Take $\sqrt{\text{MSE}}$ to get **RMSE**, which brings units back to dollars.

The Foundation: Confusion Matrix

To understand errors, we break predictions into a 2×2 table.

	Predicted YES	Predicted NO
Actual YES	TP (True Positive)	FN (False Negative)
Actual NO	FP (False Positive)	TN (True Negative)

Green = correct predictions

Red = mistakes

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Can we use accuracy as a loss function? **No** — it is non-differentiable (counts correct/wrong, no gradient).

Imagine a dataset where **99%** of samples are Class A and only **1%** are Class B.

Scenario:

- ▶ We build a “Lazy Model” that ignores the data and always predicts “**Class A**”.

The model is **useless** (never finds Class B), but Accuracy says it’s amazing. We need better metrics!

Result:

99%
Accuracy

Remember the imbalanced data problem from Day 2? This is why we introduced stratification!

Question

*“Of all the times the model predicted **YES**, how often was it right?”*

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

When to use it?

- ▶ When **False Positives** are expensive or annoying
- ▶ **Example: Spam Filter** — we do **not** want to mark a real email as spam. Better to let a few spam through than delete a real one.

Question

*“Of all the **real YES** cases in the world, how many did we find?”*

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

When to use it?

- ▶ When **False Negatives** are dangerous or fatal
- ▶ **Example: Cancer Detection** — we must find **every** sick patient. A false alarm (more tests) is okay, but missing a sick person is not.

Often we need a balance between Precision and Recall. If you optimize only one, the other usually suffers (**Precision–Recall Tradeoff**).

F1-Score (Harmonic Mean of Precision and Recall)

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why Harmonic Mean, not Arithmetic Mean?

- ▶ It forces the model to be good at **both**.
- ▶ If Precision = 100% but Recall = 0%:
 - Arithmetic Average = 50% (looks decent!)
 - **F1-Score** = 0 (correctly punishes the failure)

Which Metric to Choose?

Scenario	Priority	Metric
Spam filter	Avoid annoying users (few false positives)	Precision
Cancer screening	Catch all cases (few false negatives)	Recall
Balanced trade-off	Both matter equally	F1-Score
Balanced dataset	Overall correctness	Accuracy

Rule of thumb: On imbalanced data, **never** rely on accuracy alone.

Question: My F1-Score is 40%. Is that good?

Answer: Compare it to a “Dummy” Baseline!

Classification Baseline

- ▶ Always predict the **Majority Class**
- ▶ If your model can't beat this, it hasn't learned anything

Regression Baseline

- ▶ MSE baseline: predict the **Mean**
- ▶ MAE baseline: predict the **Median**
- ▶ Can't beat this? You're worse than “just guess the average”!

Always establish a baseline before celebrating a score.

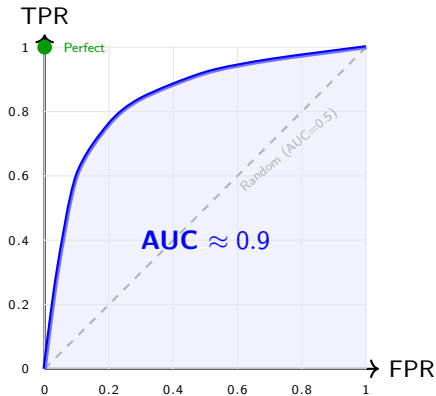
ROC = Receiver Operating Characteristic

Plots **True Positive Rate** (Recall) vs. **False Positive Rate** as the threshold varies from 0 to 1.

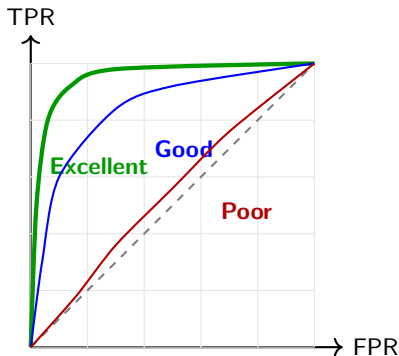
$$\text{TPR} = \frac{TP}{TP+FN} \quad (\text{Recall})$$
$$\text{FPR} = \frac{FP}{FP+TN}$$

AUC = Area Under the ROC Curve

- ▶ AUC = 1.0 perfect
- ▶ AUC = 0.5 random guess
- ▶ AUC < 0.5 worse than random



The more the curve hugs the top-left, the better the model.



AUC > 0.9

Excellent separator.
Suitable for deployment.

AUC ≈ 0.8

Good performance.
May need tuning.

AUC ≈ 0.6

Barely above random.
Needs rethinking.

	Regression	Classification
Task Output	Continuous ($y \in \mathbb{R}$)	Discrete ($y \in \{0, \dots, K-1\}$)
Loss Function	MSE / MAE	Cross-Entropy
Model	Linear Regression	Logistic Regression
Activation	Identity	Sigmoid / Softmax
Key Metrics	MSE, MAE, R^2	Accuracy, F1, AUC

Key Insight: Linear & Logistic Regression share the *same gradient formula*.
The only difference is the activation function applied to $\mathbf{w}^T \mathbf{x}$.

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program